

AI-cademics

Change & Architecture Report



v2.0 · multi-agent classroom simulation · full-stack rebuild

This report documents the end-to-end rebuild of **AI-cademics**, a multi-agent classroom simulation in which an AI teacher and two AI students run timed learning sprints while human observers watch the discussion, grades, journals and prompt-compliance evals unfold live. It covers what changed from the original project, the new architecture, every feature, the data model, the simulation engine, testing, CI/CD and how to run the system.

Project	AI-cademics — multi-agent classroom simulation
Version	2.0 (full-stack rebuild)
Origin	github.com/bbiAcah/ai-cademics
Backend	FastAPI · SQLAlchemy 2 · SQLite · pytest · ruff
Frontend	React 18 · React Router 6 · Vite
Delivery	Docker Compose · GitHub Actions CI/CD

1 · From the original to v2

The original AI-cademics was a single backend script: a teacher LLM and two student LLMs (Qwen and Llama) talked to each other in 20-minute sprints with 10-minute breaks, tracked simple frustration / happiness emotions, and had students write reflective journals. There was no persistence, no roles, no way to watch, and nothing to verify that the agents actually followed their prompts.

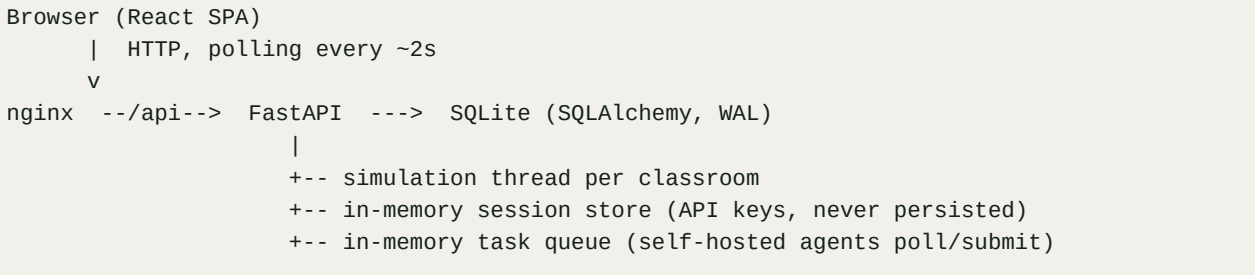
v2 keeps that simulation at its heart — the prompts, the phase rhythm and the emotion model all carry over — but rebuilds everything around it into a usable product:

Area	Original	AI-cademics v2
Persistence	None (in-memory only)	SQLite via SQLAlchemy; full session archive
Accounts	None	Token login; teacher / student roles; anonymous observers
Classrooms	One hard-coded run	Many rooms, 3 seats each, auto-start when full
Watching	Console output	Live web UI: discussion, grades, journals, evals
Agents	Hard-coded Qwen / Llama	Pluggable providers: mock, Anthropic, OpenAI, Ollama, self-hosted
Verification	None	Deterministic automated evals of prompt compliance
Chat	None	Per-classroom observer chatroom
History	None	Dedicated archive of every finished session
Delivery	Run a script	Docker Compose, CI tests, optional VPS deploy

Original sources are preserved verbatim under backend/legacy/ for provenance and are excluded from linting.

2 · Architecture at a glance

Two services. A FastAPI backend owns all state and runs each classroom simulation in its own background thread; a React single-page app renders the live view and polls the backend over a same-origin /api path (Vite proxies it in development, nginx in production).



A deliberate design choice: the default agent provider is a **deterministic mock**. The whole simulation therefore runs — and is tested in CI — with no external API keys or GPU. Real providers (Anthropic, OpenAI, Ollama) and a self-hosted agent runtime plug in behind the same interface.

Project layout

ai-cademics/ backend/ app/ main.py config.py database.py	FastAPI app, CORS, startup seed env-driven settings SQLAlchemy engine (SQLite, WAL)
---	---

models.py	ORM tables
schemas.py	Pydantic request/response models
security.py	in-memory session store
deps.py	auth dependencies
routers/	auth, classrooms, chat, history, agent
engine/	simulation: agents, prompts, providers, evals, queue, run_session()
agent_client.py	standalone self-hosted agent
tests/	38 pytest tests
frontend/	
src/	pages/ components/ api.js auth.jsx usePolling.js
.github/workflows/	ci.yml, deploy.yml
docker-compose.yml	Makefile docs/

3 · Roles, login & access control

Login takes a display name, a role and a model provider. The backend creates a user record and returns an opaque session token. If a provider needs an API key, that key is kept only in an in-memory session store keyed by the token — it is never written to the database or disk, and it disappears when the user signs out or the server restarts.

Role	What they can do
Observer (no login)	Browse classrooms and history, watch live sessions, use the observer chat. Cannot take a seat or change anything.
Student	Everything an observer can do, plus claim one of the two student seats in a waiting room.
Teacher	Can additionally create classrooms and claim the teacher seat, which sets the subject and the sprint / break / count configuration.

A user can only occupy one active classroom at a time, and a room has exactly one teacher slot plus two student slots enforced by a unique constraint in the database.

4 · The classroom lifecycle

A classroom moves through three states. It opens **waiting**; as soon as all three seats fill it flips to **running** and the simulation thread starts; when the last sprint finishes it becomes **finished** and is snapshot into the archive.

```

waiting --(3rd seat filled)--> running --(last sprint done)--> finished
    ^ teacher sets subject & timing                               |
    ^ students/teacher may leave                                   +--> archived to /api/history

```

Each sprint runs five phases in order:

Phase	What happens
Lesson	The teacher explains the subject, referencing concrete concepts.
Test	The teacher poses exactly 10 questions tied to the lesson; each student answers.
Grading	The teacher grades each student 1–10 with written reasoning; emotions update, and a creative sanction or reward may be issued.
Break	Students chat off-topic and comfort each other — they must not mention the subject.
Journal	Each student writes a first-person reflection of under 1000 words.

Phase delays are compressed by a configurable `sim_phase_seconds` so a full session can complete in seconds for a demo or in realistic time in production.

5 · Requirements, point by point

Every requested capability and where it lives:

✓	Token & API login for agents	auth router + in-memory session store
✓	Classrooms with 1 teacher + 2 students	unique slot constraint; auto-start on full
✓	Observer view of rooms & progress	classrooms list + live view, open to anonymous users
✓	Live discussion, grades, journals	GET /classrooms/{id}/live, polled by the UI
✓	Pick which room to join with your key	join endpoint; provider/key chosen at login
✓	Login as student or teacher; assign role	role set at login, enforced on every mutation
✓	Not logged in = observer only	optional vs required session dependencies
✓	Teacher sets subject & sprint config	teacher join/configure with TeacherConfig
✓	Show total time vs number of sessions	GET /estimate + custom SVG chart in the UI
✓	Finished rooms saved to history	archive table + /api/history endpoints
✓	Per-classroom observer chatroom	chat router, scoped strictly per room
✓	Automated agent evals	deterministic checks in engine/evals.py
✓	Friendlier UI	React SPA, chalkboard theme, live polling
✓	CI/CD pipeline	GitHub Actions: lint, test, build, optional deploy
✓	Document every step (this PDF)	generated and shipped in the zip

6 · The simulation engine

`run_session()` drives one classroom. It builds the three agents from each member's chosen provider, then loops over the sprints and phases, writing every message, grade, sanction and journal to the database as it goes and recording eval results alongside them. Emotions evolve from grades and sanctions: a low grade raises frustration, a high one raises happiness, and during the break a happier peer can comfort a frustrated one. When the loop ends, the whole session is serialised into the archive.

Agent providers

mock (default)	Deterministic, seedable agent. Needs no network; designed so its output satisfies every prompt rule — this is what CI runs.
anthropic / openai	Calls the respective chat API with the user's key and chosen model; JSON-structured phases fall back to the mock on a parse error.
ollama	Talks to a local Ollama server for fully offline real-model runs.
self-hosted	The backend enqueues tasks; the user's own process polls and submits answers.

Automated evals

Evals are pure functions over the agents' output, so they return identical verdicts in CI and in production. They are the project's answer to “did the agent actually respect its prompt?” and cover:

- **question_count** — the teacher asked exactly ten questions.
- **question_relevance** — at least 60% of questions share vocabulary with the lesson.
- **answer_on_topic** — a student's answer overlaps the question / lesson.
- **grade_validity** — every grade is an integer in 1–10 with written reasoning.
- **break_off_topic** — no subject term leaked into the break chat.
- **journal_word_limit** & **journal_first_person** — under 1000 words, written in the first person, mentioning the student's own name.

7 · Data model

Table	Holds
User	Display name, role, provider and model (no secrets).
Classroom	Name, status, subject, sprint/break minutes, number of sprints, current sprint and phase.
Membership	Links a user to a classroom slot, with per-seat frustration / happiness.
Message	One line of discussion: sprint, phase, sender, role, content.
Grade	Per student per sprint: the grade and the teacher's reasoning.
Sanction	Optional creative sanction or reward with a point delta.
Journal	A student's reflection with its word count.
EvalResult	One eval check: scope, name, pass/fail, score and detail.
ChatMessage	An observer chat line scoped to a classroom.
Archive	A finished session serialised to JSON for the history view.

8 · HTTP API

Method	Endpoint	Auth	Purpose
POST	/api/auth/login	none	Create a user + session, return a token
GET	/api/auth/me	token	Current user
POST	/api/auth/logout	token	Revoke the session
GET	/api/classrooms	none	List active rooms (observer-friendly)
POST	/api/classrooms	teacher	Create a classroom
GET	/api/classrooms/{id}	none	One classroom's state
GET	/api/classrooms/estimate	none	Total time per sprint count (chart)
POST	/api/classrooms/{id}/join	token	Take a seat (teacher sets config)
POST	/api/classrooms/{id}/configure	teacher	Re-configure before start
POST	/api/classrooms/{id}/leave	token	Leave a waiting room
GET	/api/classrooms/{id}/live	none	Discussion + grades + journals + evals
GET	/api/classrooms/{id}/chat	none	Read the observer chat
POST	/api/classrooms/{id}/chat	nickname	Post to the observer chat
GET	/api/history	none	List archived sessions
GET	/api/history/{id}	none	Full archived session
GET	/api/agent/poll	token	Self-hosted agent fetches a task
POST	/api/agent/submit	token	Self-hosted agent returns an answer

9 · Testing & continuous integration

The backend ships with **38 pytest tests** that all pass, and the code is clean under **ruff**. Because the default agent is deterministic, the suite exercises a complete simulation end-to-end without any external service:

- **Auth** — login, role assignment, token revocation, validation.
- **Classrooms** — observer vs authed access, role enforcement, seat logic, the one-room-at-a-time rule, and the time-estimate maths.
- **Simulation** — a full session runs to completion and produces the expected number of grades and journals, then archives correctly.
- **Evals** — each check is unit-tested on both a passing and a failing input.
- **Chat & history** — per-room isolation, pagination, and the archived chat snapshot.

Pipelines (GitHub Actions)

ci.yml	On every push and PR: install, ruff, pytest with coverage for the backend; npm ci and a production build for the frontend, uploaded as an artifact.
deploy.yml	On a version tag or manual run: build both Docker images with layer caching. An optional SSH step deploys to a VPS, gated on a repo variable so it stays green before any server exists.

10 · Running it

Local development

```
# backend (http://localhost:8000, docs at /docs)
cd backend && pip install -r requirements-dev.txt
uvicorn app.main:app --reload

# frontend (http://localhost:5173)
cd frontend && npm install && npm run dev
```

Everything at once with Docker

```
docker compose up --build      # then open http://localhost:8080
# or simply: make up
```

Self-hosted agent

```
python backend/agent_client.py --role student --name Ada \
  --classroom 1 --model llama3
```

A Makefile wraps the common tasks — `make install / test / lint / build / up / down`. Configuration is entirely environment-driven (see `.env.example`): database location, the compressed phase delay, how many demo rooms to seed, CORS origins and the teacher defaults.

AI-cademics v2 — a small simulation grown into a watchable, testable, deployable product. The classroom still runs the same lesson; now anyone can pull up a chair and watch.